

System Administration — Capstone Report

Student

Алексей Быков

Repository

<https://github.com/ab-sandbox/mephi-system-administration>

Track

База

Capstone project verification report

2. Technical Verification

2.1 Git Repository

Command

```
git log --oneline --reverse --no-decorate
```

Output

```
3def13c docs: add minimum system passport
eb430d2 docs: add base system information
119e0f9 docs: add advanced system information
a43bab6 chore: remove empty gitignore
1cfb371 docs: add SSH connection screenshot
2d6427c docs: update repository README
1d3ad73 feat: add resource monitoring script
d76c06a refactor: improve resource monitoring script
14e778d feat: add dependency checks
08cf8f2 docs: add resource monitor documentation
2b763aa refactor: organize module-1 structure
20fc9df feat: containerize resource monitor
e82634f feat: add Docker Compose configuration
6519b4c docs: document capstone setup
5c64537 feat: expose resource monitor over HTTP
95f9a88 feat: add service deployment configuration
42ec876 docs: document service deployment
edc0465 test: add capstone verification scripts
df17527 fix: make verification scripts location-independent
752cb79 docs: add capstone verification results
6bebeb3 docs: simplify repository overview
87347b2 test: add capstone verification workflow
```

Result

Git-история отражает последовательное развитие проекта: от первоначального исследования системы и разработки скрипта мониторинга до контейнеризации, запуска через Docker Compose, публикации данных по HTTP, настройки Nginx и systemd и добавления автоматизированных проверок. Изменения разделены на небольшие тематические коммиты с использованием Conventional Commits.

2.2 Resource Monitor

Command

```
ls -l ~/capstone-build/module-02/resource-monitor/script.sh
docker exec resource-monitor pgrep -af script.sh
docker exec resource-monitor tail -25 /var/www/monitor.log
```

Output

```
-rwxrwxr-x 1 ubuntu ubuntu 731 Sep 25 16:36 /home/ubuntu/capstone-build/module-02/resource-monitor/script.sh

7 bash ./script.sh

--- 2026-09-30 05:40:02 ---

[Memory]
total used free shared buff/cache available
Mem: 1.9Gi 255Mi 290Mi 1.0Mi 1.4Gi 1.5Gi
Swap: 0B 0B 0B

[Disk]
Filesystem Size Used Avail Use% Mounted on
overlay 9.6G 3.6G 6.0G 38% /
tmpfs 64M 0 64M 0% /dev
shm 64M 0 64M 0% /dev/shm
/dev/sda1 9.6G 3.6G 6.0G 38% /etc/hosts
tmpfs 979M 0 979M 0% /proc/acpi
tmpfs 979M 0 979M 0% /proc/scsi
tmpfs 979M 0 979M 0% /sys/devices/virtual/powercap
tmpfs 979M 0 979M 0% /sys/firmware

[Uptime]
05:40:02 up 2 days, 21:50, 0 users, load average: 0.01, 0.00, 0.00
```

Result

Скрипт `script.sh` запущен внутри контейнера как отдельный процесс и периодически записывает результаты мониторинга в `monitor.log`. Свежий цикл содержит сведения об использовании оперативной памяти, файловых систем и времени работы системы.

2.3 Containerization

Command

```
docker image ls resource-monitor:1.0
docker compose -f ~/capstone-build/capstone/compose.yaml ps
```

Output

```
IMAGE ID DISK USAGE CONTENT SIZE EXTRA
resource-monitor:1.0 39b758aab1b0 161MB 40.1MB U

NAME IMAGE COMMAND SERVICE CREATED STATUS
PORTS
resource-monitor resource-monitor:1.0 "/bin/bash -c './scr..." resource-monitor 2 days ago Up 40 hours
0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
```

Result

Образ `resource-monitor:1.0` успешно собран и используется запущенным контейнером. Сервис управляется через Docker Compose и публикует порт 8080 контейнера на порту 8080 хоста. На момент проверки контейнер непрерывно работал около 40 часов.

2.4 RAID 1 and LVM

Command

```
cat /proc/mdstat
sudo vgs
sudo lvs
df -h /mnt/raid1 /mnt/lvm
```

Output

```
Personalities : [linear] [multipath] [raid0] [raid1] [raid6] [raid5] [raid4] [raid10]
md0 : active raid1 loop7[2] loop6[0]
261120 blocks super 1.2 [2/2] [UU]

unused devices: <none>

VG #PV #LV #SN Attr VSize VFree
vg_data 1 1 0 wz--n- 252.00m 60.00m

LV VG Attr LSize
lv_data vg_data -wi-ao---- 192.00m

Filesystem Size Used Avail Use% Mounted on
/dev/md0 223M 32K 205M 1% /mnt/raid1
/dev/mapper/vg_data-lv_data 160M 28K 149M 1% /mnt/lvm
```

Result

Программный RAID 1 /dev/md0 собран из двух loop-устройств и находится в штатном состоянии: [2/2] [UU] показывает, что оба участника массива доступны. Массив отформатирован и смонтирован в /mnt/raid1.

На отдельном loop-устройстве создана группа томов vg_data и логический том lv_data. В ходе работы логический том был расширен со 128 MiB до 192 MiB с последующим расширением файловой системы; итоговый том смонтирован в /mnt/lvm.

2.5 Nginx Reverse Proxy

Command

```
sudo nginx -t
curl -I http://127.0.0.1/monitor.log
```

Output

```
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful

HTTP/1.1 301 Moved Permanently
Server: nginx/1.18.0 (Ubuntu)
Date: Wed, 30 Sep 2026 05:40:11 GMT
Content-Type: text/html
Content-Length: 178
Connection: keep-alive
Location: https://127.0.0.1/monitor.log
```

Result

Конфигурация Nginx успешно проходит проверку. HTTP-запрос к Nginx на стандартном порту получает ответ 301 Moved Permanently и перенаправляется на HTTPS; доступ к контейнерному сервису выполняется через настроенный reverse proxy.

2.6 TLS

Command

```
curl -kI https://127.0.0.1/monitor.log
```

Output

```
HTTP/1.1 200 OK
Server: nginx/1.18.0 (Ubuntu)
Date: Wed, 30 Sep 2026 05:40:11 GMT
Content-Type: application/octet-stream
Content-Length: 12730147
Connection: keep-alive
Last-Modified: Wed, 30 Sep 2026 05:40:02 GMT
```

Result

HTTPS-запрос через Nginx завершается ответом 200 OK, что подтверждает работу TLS и доступность ресурса `monitor.log` через reverse проxy. Для лабораторного окружения используется самоподписанный сертификат, поэтому при проверке через `curl` применяется параметр `-k`.

2.7 systemd Service

Command

```
systemctl is-enabled resource-monitor
systemctl status resource-monitor --no-pager
```

Output

```
enabled

● resource-monitor.service - Resource Monitor container
   Loaded: loaded (/etc/systemd/system/resource-monitor.service; enabled; vendor preset: enabled)
   Active: active (running) since Mon 2026-09-28 17:19:04 +04; 1 day 16h ago
     Main PID: 51703 (docker)
        Tasks: 9 (limit: 2255)
       Memory: 8.7M
          CPU: 6.015s
      CGroup: /system.slice/resource-monitor.service
              └─51703 /usr/bin/docker start -a resource-monitor

Sep 28 17:19:04 sysadmin-host systemd[1]: Started Resource Monitor container.
Sep 29 13:05:07 sysadmin-host docker[51703]: 172.18.0.1 - - [29/Sep/2026 09:05:07] "GET /monitor.log HTTP/1.0"
200 -
```

Result

Сервис `resource-monitor.service` включен в автозапуск и находится в состоянии `active (running)`. `systemd` управляет жизненным циклом контейнера через Docker и сохраняет его стандартный вывод в системном журнале.

2.8 Observability

Command

```
curl -ks https://127.0.0.1/monitor.log > /dev/null
sudo journalctl -u resource-monitor -n 10 --no-pager
sudo tail -n 5 /var/log/nginx/access.log
```

Output

```
Sep 30 09:40:12 sysadmin-host docker[51703]: 172.18.0.1 - - [30/Sep/2026 05:40:11] "HEAD /monitor.log HTTP/1.0"
200 -
Sep 30 09:40:12 sysadmin-host docker[51703]: 172.18.0.1 - - [30/Sep/2026 05:40:11] "GET /monitor.log HTTP/1.0"
200 -

127.0.0.1 - - [30/Sep/2026:09:40:11 +0400] "HEAD /monitor.log HTTP/1.1" 301 0 "-" "curl/7.81.0"
127.0.0.1 - - [30/Sep/2026:09:40:11 +0400] "HEAD /monitor.log HTTP/1.1" 200 0 "-" "curl/7.81.0"
127.0.0.1 - - [30/Sep/2026:09:40:11 +0400] "GET /monitor.log HTTP/1.1" 200 12730147 "-" "curl/7.81.0"
```

Result

Запрос к сервису фиксируется на двух уровнях: Nginx записывает обращение в `access.log`, а вывод приложения внутри контейнера доступен через журнал `systemd`. Это позволяет проследить прохождение запроса от `reverse proxy` до контейнерного HTTP-сервера.

3. Architecture Decisions

3.1 Resource Monitoring

Для проекта был выбран вариант В — периодический мониторинг ресурсов системы. Скрипт на Bash с заданным интервалом собирает сведения об использовании памяти и файловых систем, а также времени работы и текущей нагрузке системы, сохраняя результаты в `monitor.log`.

Такой вариант позволяет использовать один и тот же компонент на нескольких этапах проекта: сначала как самостоятельный системный скрипт, затем как процесс внутри Docker-контейнера и, наконец, как источник данных, доступный через HTTP и Nginx. Решение не требует дополнительных библиотек или специализированной системы мониторинга и основано на стандартных утилитах Linux.

3.2 Storage Architecture

Для демонстрации работы с хранилищем использованы две независимые конфигурации: RAID 1 и LVM. RAID 1 предназначен для повышения отказоустойчивости за счет зеркального хранения данных на двух устройствах. В ходе проверки один из участников массива был переведен в состояние отказа и удален; данные при этом оставались доступны. После возврата устройства массив был восстановлен до состояния [UU].

LVM решает другую задачу — предоставляет гибкое управление дисковым пространством поверх физических устройств. Созданный логический том `lv_data` был расширен со 128 MiB до 192 MiB, после чего файловая система была увеличена без потери существующих данных.

В лабораторной среде RAID и LVM реализованы независимо друг от друга на loop-устройствах. Это позволяет отдельно продемонстрировать отказоустойчивость RAID и возможность изменения размера хранилища средствами LVM без усложнения учебной конфигурации.

3.3 TLS Certificate

Для HTTPS используется самоподписанный TLS-сертификат. Такой вариант выбран потому, что сервис развернут в изолированной лабораторной виртуальной машине и не имеет публичного доменного имени, для которого требовался бы сертификат от доверенного центра сертификации.

TLS завершается на Nginx, который принимает HTTPS-соединение на порту 443 и передает запрос контейнерному сервису по HTTP на `127.0.0.1:8080`. Это отделяет настройку шифрования от приложения и позволяет контейнеру оставаться простым HTTP-сервисом.

При проверке используется `curl -k`, поскольку самоподписанный сертификат отсутствует в доверенном хранилище клиента. Этот параметр отключает проверку доверия к сертификату, но само TLS-соединение при этом остается зашифрованным.

3.4 Alternatives Considered

Вместо самоподписанного сертификата можно было использовать сертификат от публичного центра сертификации, например Let's Encrypt. Для локального лабораторного сервиса без публичного доменного имени это добавило бы внешние зависимости и не дало преимуществ для демонстрации TLS, поэтому был выбран самоподписанный сертификат.

Контейнерный HTTP-сервис можно было опубликовать напрямую через порт 8080. Использование Nginx добавляет отдельный reverse проху, на котором централизованы перенаправление HTTP на HTTPS и завершение TLS, при этом само приложение остается простым HTTP-сервисом.

Для автоматического перезапуска контейнера можно было использовать restart policy Docker Compose. В проекте управление контейнером передано systemd, чтобы интегрировать сервис в стандартный механизм управления службами Linux и обеспечить его запуск вместе с системой.

4. Troubleshooting

Во время настройки Nginx проверка конфигурации первоначально сообщала о конфликте `server_name` с конфигурацией сайта по умолчанию. Стандартный сайт был отключен удалением ссылки `/etc/nginx/sites-enabled/default`, после чего `nginx -t` завершился успешно.

При проверке отказоустойчивости RAID 1 один из участников массива был намеренно переведен в состояние отказа и удален. Массив перешел из `[UU]` в деградированное состояние `[U_]`, однако данные остались доступны для чтения и записи. После повторного добавления устройства массив синхронизировался и вернулся в состояние `[UU]`.

Первоначальная версия verification-скрипта для Docker Compose зависела от текущего рабочего каталога и не находила `capstone/compose.yaml` при запуске из каталога `verify`. Путь был исправлен путем определения расположения самого скрипта через `BASH_SOURCE` и вычисления `CAPSTONE_DIR`, после чего проверка стала независимой от каталога запуска.